

PKCERT AI & Software Development Internship

Task 27: NLP Basics – Tokenization, Word Embeddings, and Hugging Face Transformers

Abdullah Amir

AI and Software Development
17 August 2026

Full code lives in `Day_27.ipynb` and the standalone scripts it is built from. **Pure Python, no external tokenizer library**, for Part A's from-scratch BPE implementation, per the brief's explicit restriction for that sub-task – **gensim** (Word2Vec), pretrained **GloVe** vectors, and **Hugging Face transformers** (DistilBERT) for Parts B/C, none reimplemented from scratch. Random seed **42** throughout.

Part A: Tokenization Foundations

A1 – word/character/subword trade-offs. **Word-level:** short sequences, semantically meaningful tokens; unbounded vocabulary and total-information-loss OOV handling (every unseen word collapses to one `<unk>`). **Character-level:** tiny, fixed vocabulary, near-zero true OOV; long sequences, low per-token semantic content, harder credit assignment. **Subword-level:** the practical middle ground – frequent whole words stay single tokens, rare/unseen words decompose into frequent pieces, bounding vocabulary while keeping OOV words *meaningfully* represented rather than discarded.

A2 – BPE, derived. Represent each corpus word as characters + an end-of-word marker `</w>` (prevents merges spanning word boundaries); initialize the vocabulary as all individual characters; repeatedly count every adjacent symbol-pair's frequency (weighted by word frequency), merge the single most frequent pair everywhere it occurs, and record the merge. Encoding a new word applies the learned merges, in order, to its character sequence – any substring matching no merge stays as individual characters, so BPE always produces *something* meaningful for a novel word, never a single `<unk>`.

A4 – from-scratch implementation. SimpleBPE, trained on a 6-sentence corpus, 40-merge budget:

Metric	Value
Final vocabulary size	67
Merges performed	40
Sample encode→decode round-trip	lossless

Sample sentence "the lower price of the newest fox" encodes to 12 tokens and decodes back *exactly*. The out-of-vocabulary word "slowest" (never seen in training) encodes to `['s', 'low', 'e', 'st</w>']` – reusing the "low" piece learned from "lower"/"low", a concrete demonstration of A1's subword-OOV advantage.

A3 – BPE vs. WordPiece vs. SentencePiece/Unigram. All three learn subword vocabularies from a corpus; they differ in the merge/unit-selection criterion. **BPE** (GPT-2/3/4-family): merges the pair with highest *raw frequency*. **WordPiece** (BERT, DistilBERT): merges the pair maximizing training-corpus *likelihood* under a unigram LM, scored by $\text{count}(ab)/(\text{count}(a) \cdot \text{count}(b))$.

count(b)) – prefers pairs that co-occur more than their individual frequencies alone predict. **SentencePiece/Unigram** (T5, ALBERT, XLNet): top-down pruning from a large candidate set, removing whichever unit costs least corpus log-likelihood to drop – likelihood-based like WordPiece but prune rather than merge, and typically operates on raw bytes/characters including whitespace, making it language-agnostic for languages without clear word boundaries.

A5 – preprocessing. Lowercasing trades case information for smaller vocabulary (uncased vs. cased pretrained variants). Unicode normalization (NFC/NFKC) ensures visually-identical text produces identical tokens regardless of underlying byte representation. Punctuation handling affects whether patterns like contractions generalize across many words or must be seen independently per form. Special tokens ([CLS]/[SEP]/[PAD]/[UNK]) are inserted into the token sequence itself: [CLS]’s final hidden state conventionally summarizes the whole sequence (Task 26’s classification head reads exactly this position); [SEP] marks segment boundaries; [PAD] must be excluded from attention via the attention mask; [UNK] is subword tokenization’s rarely-needed fallback of last resort.

Part B: Word Embeddings – Word2Vec & GloVe

B1 – distributional hypothesis; sparse vs. dense. “A word is characterized by the company it keeps” (Firth, 1957) motivates learning representations from co-occurrence statistics. One-hot/BoW/TF-IDF are structurally unable to express similarity (any two distinct one-hot vectors are orthogonal); dense embeddings place distributionally similar words close together in a low-dimensional continuous space, directly operationalizing the hypothesis.

B2 – Skip-gram, softmax, negative sampling.

$$J(\theta) = \frac{1}{T} \sum_t \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t), \quad P(w_O | w_I) = \frac{\exp(u_{w_O}^\top v_{w_I})}{\sum_{w=1}^{|V|} \exp(u_w^\top v_{w_I})}$$

The softmax denominator’s $O(|V|)$ cost per training pair is intractable at real vocabulary scale. **Negative sampling** replaces it with a binary classification task – true pair vs. k sampled noise words:

$$J_{\text{neg}} = -\log \sigma(u_{w_O}^\top v_{w_I}) - \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-u_{w_i}^\top v_{w_I})]$$

The first term pulls the true context vector toward the center word; the second (via $\sigma(-x) = 1 - \sigma(x)$) pushes sampled negative words’ vectors away – reducing per-step cost from $O(|V|)$ to $O(k)$.

B3 – Skip-gram vs. CBOW. Skip-gram predicts context *from* center (up to $2m$ separate training pairs per window); CBOW predicts center *from* averaged context (one prediction per window, generally faster per corpus pass). Skip-gram represents *rare* words better – every context position around a rare center word still yields its own gradient update, whereas CBOW’s averaging dilutes any single rare context word’s contribution among its neighbors.

B4 – GloVe. Co-occurrence matrix X_{ij} = corpus-wide count of word j within word i ’s context window. Log-bilinear model $w_i^\top \tilde{w}_j + b_i + \tilde{b}_j = \log X_{ij}$, fit via weighted least squares:

$$J = \sum_{i,j=1}^{|V|} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

with f zero at $X_{ij}=0$, non-decreasing, but saturating for very frequent pairs. GloVe combines global statistics (the co-occurrence matrix is aggregated once, corpus-wide) with Word2Vec’s

local-context-window notion of what counts as “context” in the first place – matrix factorization’s efficient use of global statistics, plus local-window prediction’s tendency toward strong linear analogical structure.

B5/B6 – trained Word2Vec (gensim, AG News) vs. pretrained GloVe.

	Word2Vec (self-trained)	GloVe (pretrained)
Training corpus	6,000 AG News docs, 232K tokens	6B tokens (Wikipedia+Gigaword)
Vocabulary	5,617	400,000

Analogy	Word2Vec result	GloVe result
king – man + woman	wing (wrong)	queen (correct)
paris – france + germany	producers (wrong)	berlin (correct)
company – ceo + country	companies (n/a)	nation (partial)

Self-trained Word2Vec shows **strong topical nearest-neighbor clustering** despite its tiny corpus – its nearest neighbors for “president” are **hamid, hugo, karzai, chavez**, literally real president names from AG-News-era headlines – but **fails every analogy tested**. GloVe succeeds at both. This is a corpus-size effect, not an algorithmic one: 232K tokens is $\sim 25,000\times$ smaller than GloVe’s 6B-token corpus, and analogical linear structure is far more data-hungry to emerge than simple topical co-occurrence similarity.

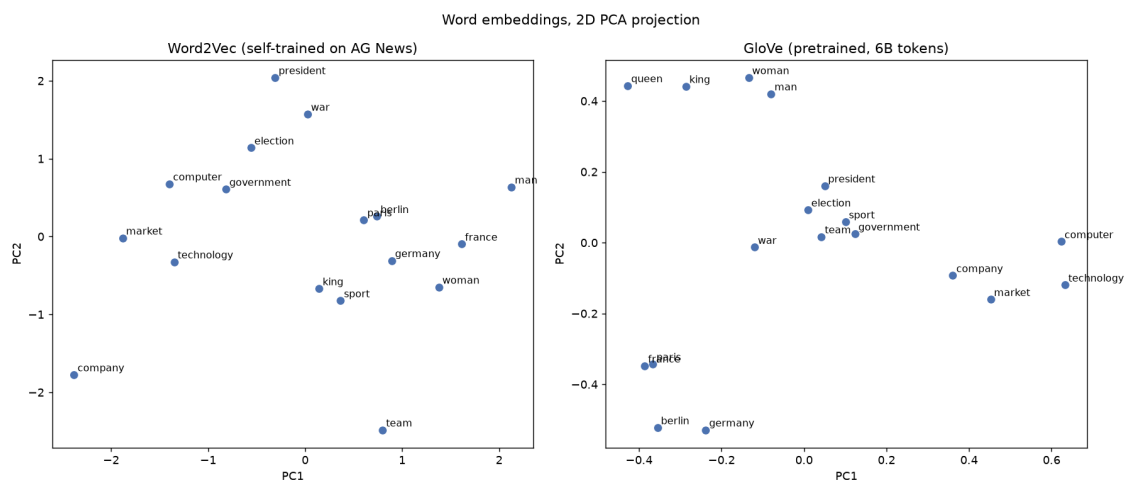


Figure 1: 2D PCA projection: Word2Vec (self-trained) vs. GloVe (pretrained).

The visualization confirms this directly: GloVe’s king/queen/man/woman and paris/france/berlin/germany clusters are tight and well-separated; Word2Vec’s corresponding points are visibly noisier and less well-organized.

Part C: Applied Hugging Face Transformers Analysis

C1 – tokenizer contrast. DistilBERT’s WordPiece tokenizer produces 13 tokens (including [CLS]/[SEP]) for a 10-word sample sentence; Part A’s toy BPE (6-sentence training corpus vs. billions of words) produces 24 tokens on the identical sentence, decomposing most words into

short character fragments it never learned as merged units. Same algorithm family (subword merge-based), vastly different training scale.

C2 – static embeddings vs. Word2Vec/GloVe.

Pair	DistilBERT (static)	Word2Vec	GloVe
king-queen	0.657	n/a*	0.751
king-car	0.250	0.303	0.283
man-woman	0.650	0.462	0.832
company-team	0.415	0.152	0.461

Table 1: *“queen” fell below Word2Vec’s min_count=5 frequency threshold in the small AG News subset.

Different vector spaces/dimensionalities mean only the *pattern* is comparable, not raw values – all three models agree king-queen and man-woman are markedly more similar than king-car, a consistent qualitative signal across independently-trained embedding spaces.

C3 – contextual embeddings, polysemy demo. The word “bank” extracted from DistilBERT’s final hidden layer across three sentences:

Sentence pair	Cosine similarity
financial_1 vs. financial_2 (same sense)	0.851
financial_1 vs. geographic (different sense)	0.727
geographic vs. financial_2 (different sense)	0.623

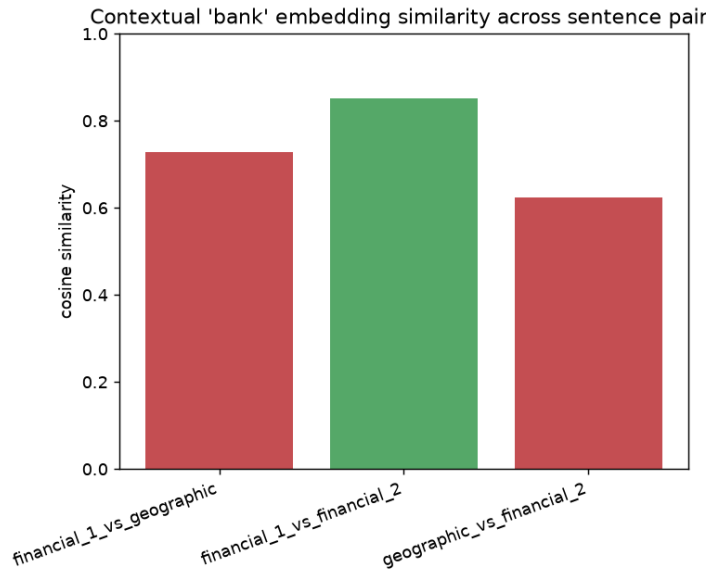


Figure 2: Contextual “bank” embedding similarity across sentence pairs.

Same-sense similarity (0.851) exceeds both different-sense pairs – a measurable, quantitative confirmation that contextual embeddings shift representation by word sense, which a fixed Word2Vec/GloVe vector (Part B) structurally cannot do: one vector per word, identical regardless of sentence.

C4 – OOV handling contrast. WordPiece decomposes any unseen word into known sub-word/character pieces – there is no true OOV event at the tokenizer level, so the embedding lookup always has a valid row to retrieve. Classical Word2Vec/GloVe have a *closed* vocabulary fixed at training time: a word absent from (or below `min_count` in) the training corpus has **no vector at all**, not a degraded approximation – exactly what happened to “queen” in Part C2’s table above.

C5 – pipeline connection to Task 26. The tokenizer converts text to token ids (no learned parameters); the embedding layer maps ids to dense vectors via a learned lookup table – the *first* learned computation in the pipeline, and exactly the X matrix Task 26’s $Q=XW^Q, K=XW^K, V=XW^V$ projections are computed from. Positional encoding (Task 26 Part B2) is added directly to these same vectors, since self-attention has no inherent notion of order. Every subsequent attention layer builds progressively more contextualized representations on top of this initial embedding – exactly what Part C3 observed directly: the same input embedding for “bank” becomes sense-differentiated only *after* passing through attention layers, not at the embedding-lookup stage itself.

Part D: Analysis & Documentation

D1 – pipeline summary. Tokenization (A) bounds vocabulary while preserving meaningful OOV representation; sparse-vs-dense (B1) motivates embeddings amenable to similarity arithmetic; Word2Vec (B2/B3) and GloVe (B4) are two paradigms for learning those embeddings from corpus statistics, contrasted empirically in B5/B6; Hugging Face tokenizer/embedding extraction (C) demonstrates the same conceptual pipeline at production scale, composed with self-attention (Task 26) to become contextual rather than static – directly bridging this task’s classical-NLP foundations to Task 26’s applied Transformer analysis.

D2 – quantitative findings. See B5/B6 and C2/C3’s tables above: Word2Vec wins on topical nearest-neighbors despite a tiny corpus but fails analogies; GloVe succeeds at both given a $\sim 25,000\times$ larger corpus; DistilBERT’s static embeddings agree qualitatively with both on relative word-pair similarity; DistilBERT’s contextual embeddings measurably separate word senses (0.851 same-sense vs. 0.727/0.623 different-sense), which static embeddings cannot do by construction.

D3 – two implementation challenges. (1) Early BPE testing risked merges spanning word boundaries without the `</w>` end-of-word marker – resolved by appending it to every word before pair-counting, then manually verifying the merge log (`"low"</w>`, `"new"+er`) contained only genuine within-word merges. (2) `gensim` failed to build from source under this machine’s default Python 3.14 (a `PyLongObject/ob_digit` Cython-internals error from extensions compiled against older CPython APIs) – diagnosed by reading the actual compiler error rather than assuming a requirements-version mismatch, resolved by creating this task’s virtual environment with `python3.11` instead, under which `gensim` installs from a prebuilt wheel.

D4 – reflection. Static embeddings’ limitations are demonstrated concretely, not just asserted: polysemy (C3’s measured 0.851 vs. 0.727/0.623 similarity gap), fixed vocabulary (C2’s “queen” OOV event for Word2Vec), and general context-insensitivity. A lightweight static-embedding approach remains preferable for **resource-constrained or latency-critical deployment**: a Word2Vec/GloVe lookup is a single $O(1)$ array index, versus a Transformer’s $O(n^2d)$ self-attention (Task 26 Part B5) repeated across every layer just to produce one contextual embedding. For applications where polysemy resolution isn’t essential – a keyword-based document router, coarse topic-similarity search over short queries, or an edge device with no practical way to run even a distilled Transformer – a static embedding table costs orders of magnitude

less memory/compute for comparable value, and remains more directly interpretable: a fixed per-word vector can be inspected and clustered in isolation (Part B7's visualization) in a way a context-dependent vector, a different point in space for every sentence, cannot be.